

```
import streamlit as st

import streamlit.components.v1 as components

from typing import Any, Dict, Optional


from utils.db import get_supabase
from utils.razorpay import get_razorpay_client, get_razorpay_keys
from subscriptions.utils import plan_name


# -----
# SAFE HELPERS
# -----

def safe_dict(value: Any) -> Optional[Dict[str, Any]]:
    return value if isinstance(value, dict) else None


def safe_int(value: Any) -> int:
    if isinstance(value, int):
        return value
    if isinstance(value, float):
        return int(value)
    if isinstance(value, str) and value.isdigit():
        return int(value)
    return 0
```

```
def safe_str(value: Any) -> str:
```

```
    return str(value) if isinstance(value, (str, int, float)) else ""
```

```
def safe_order_create(client: Any, payload: Dict[str, Any]) -> Dict[str, Any]:
```

```
    """Safe wrapper for Razorpay order.create()"""
```

```
    try:
```

```
        order = client.order.create(payload) # type: ignore[attr-defined]
```

```
        return order if isinstance(order, dict) else {}
```

```
    except Exception as exc:
```

```
        st.write("DEBUG: Exception in safe_order_create:", exc)
```

```
        return {}
```

```
# -----
```

```
# MAIN CHECKOUT PAGE
```

```
# -----
```

```
def render_razorpay_checkout() -> None:
```

```
    st.write("DEBUG: render_razorpay_checkout() called")
```

```
    sb = get_supabase()
```

```
    # Require logged-in user
```

```
    user_res = sb.auth.get_user()
```

```
    user = user_res.user if user_res and getattr(user_res, "user", None) else None
```

```

if not user:

    st.error("You are not logged in.")

    st.stop()

# -----

# READ SUBSCRIPTION ID

# -----

sub_id = st.session_state.get("sub_id", "")

if not sub_id:

    params = st.query_params

    sub_id = params.get("sub_id", [""])[0]

st.write("DEBUG: final sub_id used in checkout =", sub_id)

if not sub_id:

    st.error("Missing subscription ID.")

    st.stop()

# -----

# FETCH SUBSCRIPTION

# -----

res = (

    sb.table("subscriptions")

    .select("*")

    .eq("id", sub_id)

```

```

        .single()
        .execute()
    )
    st.write("DEBUG: subscription fetch response =", res)

    sub = safe_dict(res.data if res else None)
    st.write("DEBUG: subscription record in checkout =", sub)

    if not sub:
        st.error("Subscription not found.")
        st.stop()

    # Authorization check
    if safe_str(sub.get("user_id")) != getattr(user, "id", ""):
        st.error("Unauthorized access.")
        st.stop()

    # Extract fields safely
    amount = safe_int(sub.get("amount"))
    currency = safe_str(sub.get("currency")) or "INR"
    plan_code = safe_str(sub.get("plan_code"))
    plan = plan_name(plan_code)

    st.write("DEBUG: checkout fields:", {
        "amount": amount,
        "currency": currency,

```

```

        "plan_code": plan_code,
        "plan": plan,
    })

# -----

# FREE PLAN HANDLING

# -----

if amount == 0:

    st.success("You have subscribed to the free plan!")

    sb.table("subscriptions").update({

        "status": "active",

        "payment_status": "free",

        "razorpay_order_id": None,

        "razorpay_payment_id": None,

        "razorpay_signature": None,

    }).eq("id", sub_id).execute()

    st.switch_page("pages/subscriptions.py")

    return

# -----

# PAID PLAN — RAZORPAY ORDER CREATION

# -----

client = get_razorpay_client()

st.write("DEBUG: Razorpay client =", client)

```

```
if client is None:
```

```
    st.error("Razorpay keys not configured.")
```

```
    st.stop()
```

```
key_id, key_secret = get_razorpay_keys()
```

```
st.write("DEBUG: key_id =", key_id)
```

```
st.write("DEBUG: key_secret length =", len(key_secret))
```

```
order = safe_order_create(
```

```
    client,
```

```
    {
```

```
        "amount": amount,
```

```
        "currency": currency,
```

```
        "payment_capture": 1,
```

```
        "notes": {
```

```
            "subscription_id": sub_id,
```

```
            "user_id": getattr(user, "id", ""),
```

```
            "plan_code": plan_code,
```

```
        },
```

```
    },
```

```
)
```

```
st.write("DEBUG: Razorpay order response =", order)
```

```
order_id = safe_str(order.get("id"))
```

```
st.write("DEBUG: order_id =", order_id)
```

```
if not order_id:
```

```
    st.error("Failed to create Razorpay order.")
```

```
    st.stop()
```

```
# Save order_id
```

```
sb.table("subscriptions").update({
```

```
    "razorpay_order_id": order_id
```

```
}).eq("id", sub_id).execute()
```

```
# -----
```

```
# RENDER RAZORPAY POPUP
```

```
# -----
```

```
html = f"""
```

```
<html>
```

```
<body>
```

```
<script src="https://checkout.razorpay.com/v1/checkout.js"></script>
```

```
<script>
```

```
    var options = {{
```

```
        "key": "{key_id}",
```

```
        "amount": "{amount}",
```

```
        "currency": "{currency}",
```

```
        "name": "Math Hub",
```

```
        "description": "{plan}",
```

```
        "order_id": "{order_id}",
```

```

    "handler": function (response) {{
        window.location.href =
            "?page=subscriptions"
            + "&payment_id=" + response.razorpay_payment_id
            + "&order_id={order_id}"
            + "&signature=" + response.razorpay_signature;
    }},
    "theme": {{
        "color": "#00ff88"
    }}
    }};

    var rzp = new Razorpay(options);
    rzp.open();
</script>
</body>
</html>
""""

```

```

components.html(html, height=10)

st.write("DEBUG: Razorpay popup should now be open")
math_saas/pages/razorpay_checkout.py and auth.py code:
import streamlit as st
import re
from typing import Any, Dict, List, cast
from utils.db import get_supabase
from themes.theme import is_dark_theme

```



```
TEXT_MUTED = "#a0a6b1"
```

```
TEXT_MAIN = "#f8f9fa"
```

```
ACCENT = "#00ff88"
```

```
# -----
```

```
# GLOBAL STYLE
```

```
# -----
```

```
def app_container_style():
```

```
    dark = is_dark_theme()
```

```
    if dark:
```

```
        bg = "radial-gradient(circle at 20% 20%, #0f1115, #050608 60%)"
```

```
        text = "#f8f9fa"
```

```
        card_bg = "rgba(18, 20, 23, 0.65)"
```

```
        card_border = "rgba(0,255,136,0.25)"
```

```
        card_shadow = "0 0 22px rgba(0,255,136,0.18)"
```

```
        accent = "#00ff88"
```

```
    else:
```

```
        bg = "#ffffff"
```

```
        text = "#222222"
```

```
        card_bg = "#fafafa"
```

```
        card_border = "#ddd"
```

```
        card_shadow = "0 0 12px rgba(0,0,0,0.08)"
```

```
        accent = "#009944"
```

```
st.markdown(
    f"""
    <style>
    body, .stApp {{
        background: {bg} !important;
        color: {text} !important;
    }}

    p, span, label, h1, h2, h3, h4, h5, h6, div {{
        color: {text} !important;
    }}

    .neon-card, .app-card {{
        background: {card_bg} !important;
        border-radius: 14px;
        padding: 20px;
        border: 1px solid {card_border} !important;
        box-shadow: {card_shadow} !important;
    }}

    h1, h2, h3, h4 {{
        color: {accent} !important;
    }}
    </style>
    """,
    unsafe_allow_html=True,
```

)

SANITIZERS

def sanitize_html(text: str) -> str:

if not isinstance(text, str):

return ""

text = re.sub(r"</?div[^>]*>", "", text, flags=re.IGNORECASE)

return text.replace(" ", " ").strip()

def clean_math(text: Any) -> str:

if not isinstance(text, str):

return ""

text = text.replace("\\\\(", "\\(").replace("\\\\)", "\\)")

text = text.replace("\\\\[", "\\[").replace("\\\\]", "\\]")

text = text.replace("\\\\frac", "\\frac")

return re.sub(r"\\$\\$s*\\$", "\$\$", text)

PUBLIC CONTENT

```
def render_public_content() -> None:
```

```
    sb = get_supabase()
```

```
    res = (
```

```
        sb.table("public_content")
```

```
        .select("*")
```

```
        .order("created_at", desc=True)
```

```
        .execute()
```

```
    )
```

```
    raw_items = getattr(res, "data", []) or []
```

```
    items: List[Dict[str, Any]] = [
```

```
        cast(Dict[str, Any], row) for row in raw_items if isinstance(row, dict)
```

```
    ]
```

```
    if not items:
```

```
        st.info("No public content available.")
```

```
        return
```

```
    for item in items:
```

```
        title = str(item.get("title", "Untitled"))
```

```
        body_raw = item.get("body", "")
```

```
        body = sanitize_html(clean_math(body_raw))
```

```
        st.markdown(f"### {title}")
```

```
        st.markdown(body, unsafe_allow_html=True)
```

```

# -----
# DISPLAY NAME HANDLER (Fix for Pydantic User)
# -----

def get_display_name(user: Any, role: str) -> str:
    # Case 1: dict (from profiles table)
    if isinstance(user, dict):
        return (
            user.get("full_name")
            or user.get("name")
            or user.get("email")
            or role.capitalize()
        )

    # Case 2: Supabase User object (Pydantic)
    meta = getattr(user, "user_metadata", {}) or {}
    email = getattr(user, "email", None)

    return (
        meta.get("full_name")
        or meta.get("name")
        or email
        or role.capitalize()
    )

```

```

# -----
# TOP BAR (Refactored)
# -----

def top_bar(title: str, role: str, logout_param: str):
    dark = is_dark_theme()

    if dark:
        bar_bg = "#0a0c10"
        bar_border = "#00ff88"
        name_color = "#00ff88"
        title_color = "ffffff"
    else:
        bar_bg = "#f0f0f0"
        bar_border = "cccccc"
        name_color = "#009944"
        title_color = "#222222"

    user = st.session_state.get("user")
    display_name = get_display_name(user, role)

    st.markdown(
        f"""
        <div style="
            background:{bar_bg};
            border-bottom:1px solid {bar_border};
            padding:12px 16px;

```

```
        border-radius:6px;
    ">
        <span style="color:{name_color}; font-weight:600;">{display_name}</span>
        <h3 style="margin:4px 0 0 0; color:{title_color};">{title}</h3>
    </div>
    """,
    unsafe_allow_html=True,
)
```

```
if st.button("Logout", key=f"logout_{role}"):
    logout()
```